

Aplicación Integral de Pruebas de Software Automatizadas en TechNext Solutions.

Introducción

Las pruebas de software son la actividad sistemática destinada a **detectar defectos** en un producto antes de que estos lleguen a producción. Como recuerda la literatura académica y técnica, el propósito del testing no es demostrar que el software no falla, sino **encontrar fallos de forma proactiva** mediante técnicas y procedimientos reproducibles. Estas prácticas son especialmente críticas en empresas que entregan soluciones multicomponente y multilenguaje, donde fallos sutiles en integración o límites de dominio pueden provocar pérdidas económicas o afectar la experiencia del usuario.

TechNext Solutions, proveedor internacional de una suite SaaS para automatización empresarial, emprendió un proyecto de transformación cuyo objetivo central fue elevar la calidad mediante la **aplicación coherente, trazable y automatizada** de pruebas: desde el diseño de casos hasta la ejecución en pipelines CI/CD y la gestión documental en TestLink. El presente caso se concentra en **cómo** se aplicaron las técnicas, qué errores concretos se detectaron con cada técnica/herramienta y qué impacto operativo y financiero tuvo la intervención.

Contexto Empresarial

TechNext Suite integra tres componentes principales:

- Backend (C# / .NET Core) que expone APIs REST;
- Procesamiento de datos (Python) para ETL y transformaciones;
- Frontend (React / JavaScript) para la experiencia de usuario.

El desarrollo era distribuido (equipos en Nicaragua, México y España), y el crecimiento orgánico había creado deuda técnica y falta de uniformidad en prácticas de verificación. La empresa observó que muchos defectos eran **errores de integración y regresión**,

y que la cobertura de pruebas era insuficiente para un producto crítico para clientes del sector financiero. La dirección aprobó un plan que convertía a las pruebas en eje operacional y no en actividad marginal.

Diseño del Plan de Pruebas y Estrategia de Aplicación

Se creó un plan de pruebas formal (objetivos, criterios de entrada/salida, recursos, responsabilidades). La estrategia siguió el **modelo en V**, asignando tipos y niveles de prueba: unitarias, integración, sistema y aceptación. Para el diseño de casos se adoptaron técnicas bien definidas: partición de equivalencia, análisis de valores límite, tablas de decisión y pruebas de transición de estados para flujos con máquinas de estados (muy útiles en protocolos de pago como MDB/ICP). Estas técnicas, descritas en profundidad por Sánchez Peño, permiten optimizar el conjunto de casos y aumentar la probabilidad de encontrar defectos relevantes.

Se decidió que TestLink sería la herramienta documental para **vincular requisitos, casos y ejecuciones**, garantizando trazabilidad, auditoría y cumplimiento contractual (instalación y uso en servidor local conforme guía práctica).

Herramientas y su aplicación práctica

- **Análisis estático:** SonarQube y linters (detectaron vulnerabilidades de estilo, variables no usadas y posibles NPEs) — importante para reducir defectos antes de ejecutar pruebas dinámicas.
- **Unit testing:** xUnit (C#), pytest (Python), Jest (JS) — integrados en GitHub Actions.
- **Automatización UI:** Selenium WebDriver (scripts en Python y JS) para probar los flujos críticos de usuario (login, pagos, generación de reportes).

- **Gestión de pruebas:** TestLink para planificar suites, asociar cada caso a un requisito y almacenar resultados de ejecución. La guía práctica de instalación y uso de TestLink fue aplicada para crear proyectos y test suites.
- **Pruebas de carga:** JMeter para escenarios concurrentes y detección de degradación de rendimiento.
- **Simuladores / drivers & stubs:** se emplearon drivers y stubs para simular pasarelas externas y terminales TPV cuando era costoso o inseguro realizar pruebas con sistemas reales.

Implementación práctica — errores detectados y corregidos

A continuación se describen **errores concretos** que aparecieron en TechNext y **cómo** fueron detectados por pruebas específicas, qué técnica permitió su diseño y qué tecnología lo confirmó o previno:

1) Error de redondeo monetario en Backend (C#) — detectado con pruebas unitarias y análisis de límites

Síntoma: algunos pagos de clientes en Nicaragua mostraban diferencias de 0.01 en reportes contables.

Causa: uso inapropiado de tipos double en cálculos de comisiones (problema de coma flotante) y falta de pruebas de valor límite que cubrieran redondeos a 2 decimales con distintas configuraciones regionales (separador decimal y redondeo bancario).

Técnica aplicada: análisis de valores límite y partición de equivalencia para identificar clases de entrada (montos bajos, medianos, altos, máximos permitidos).

Cómo se detectó: pruebas unitarias xUnit con casos que verificaban resultados usando decimal y reglas de redondeo “bankers rounding”. Los tests fueron incluidos en pipeline CI; cuando se cambió un cálculo el PR falló hasta ajustar el uso de decimal y la lógica de

redondeo.

Impacto: evitó discrepancias contables que habrían causado reclamos y sanciones fiscales locales en Nicaragua. Resultado: ahorro estimado (evitado) de horas de conciliación y reputación.

2) Regresión visual en UI — botón “Confirmar” desaparecía en resoluciones móviles — detectado por Selenium

Síntoma: en ciertos navegadores móviles (Android WebView) el botón de confirmación no se mostraba, impidiendo completar transacciones.

Causa: cambio en CSS responsivo que ocultó el contenedor en condiciones CSS combinadas. No había casos de prueba automatizados para ese breakpoint.

Técnica aplicada: pruebas de caso de uso y pruebas exploratorias, más pruebas de caja negra enfocadas a compatibilidad y usabilidad móvil.

Cómo se detectó: Selenium ejecutó scripts en emulación de pantallas móviles en el pipeline; el test UI de “flujo de compra” falló, generando screenshot almacenado en TestLink y referente en el ticket de incidencia.

Impacto: se corrigió en menos de 2 horas tras bloqueo del PR; evitó pérdidas de conversión en mercados de LATAM donde el tráfico móvil es predominante.

3) Race condition en procesamiento de lotes (Python) — descubierto por pruebas de carga y estrés

Síntoma: ocasionalmente los reportes agrupados mostraban registros duplicados o pérdidas. Ocurrió bajo alta concurrencia (procesamiento nocturno con múltiples hilos).

Causa: funciones no idempotentes y acceso concurrente a una tabla temporal sin locking adecuado. No se habían definido stubs ni drivers para simular colas altas durante pruebas unitarias/integración.

Técnica aplicada: pruebas dinámicas de carga + pruebas de integración con drivers

(stubs) que replicaron colas de mensajes. También se usó análisis de condiciones y rutas (caja blanca) para identificar puntos críticos.

Cómo se detectó: JMeter combinado con ejecuciones de pytest con escenarios de múltiples hilos provocó la aparición del fallo repetible; corrección: asegurar transacciones atómicas y uso de locks distribuidos.

Impacto: evitar interrupciones en la conciliación nocturna de clientes financieros en Managua; mejoró la estabilidad de batchs y redujo reclamos operativos.

4) Incompatibilidad con pasarela de pagos local (formato de fecha) — pruebas de integración y TestLink

Síntoma: rechazo de transacciones por formato de fecha (DD/MM/YYYY vs MM/DD/YYYY) en gateway local de Nicaragua.

Causa: asunción de formato US en serialización de payloads JSON; no se incluyeron casos de aceptación con formatos regionales.

Técnica aplicada: pruebas de aceptación y tablas de decisión para formatos de fecha, y pruebas de integración con stubs del proveedor de pagos.

Cómo se detectó: caso de prueba de aceptación en TestLink marcó fallas al ejecutar integración; se creó test suite específico y se automatizó mediante pytest+requests contra el simulador.

Impacto: evitó rechazos en producción y multas por incumplimiento de SLA; además generó una política interna de internacionalización (i18n) y normalización de fechas.

5) Fuga de memoria en servicio de Streaming (detected by load tests & monitoring)

Síntoma: consumo de RAM incrementaba gradualmente hasta reinicios del servicio bajo carga prolongada.

Causa: colecciones no limpiadas y referencias retenidas por cachés en Python.

Técnica aplicada: pruebas de esfuerzo y monitorización (profilers) combinadas con análisis estático y revisión de código.

Cómo se detectó: ejecución prolongada con JMeter y monitorización en entorno de staging, reproduciendo 72 horas de carga equivalente a pico de mes. Se empleó Wireshark y herramientas de monitoreo para correlacionar consumo con patrones de entrada. Resultado: refactor del servicio y adición de pruebas de humo que detectan consumo anómalo.

6) Implementación errónea del protocolo MDB/ICP en el TPV — pruebas funcionales y TestLink

Síntoma: operaciones REVALUE no completaban correctamente en máquinas expendedoras integradas; el TPV respondía un ACK erróneo en determinadas secuencias.

Causa: interpretación incompleta del estado de sesión del protocolo MDB/ICP (se omitía transición "Session Complete" en casos límite).

Técnica aplicada: pruebas de transición de estado y casos funcionales derivados del protocolo MDB/ICP; uso de TestLink para documentar las pruebas y ejecutar simulaciones con un VMC (simulador).

Cómo se detectó: se diseñó una test suite (MDB-25) en TestLink que replicaba exactamente la secuencia de comandos VEND/REVALUE; la ejecución automatizada contra el simulador del TPV reprodujo el fallo y permitió su corrección.

Impacto: evitó fallos en terminales desplegadas en puntos de venta en Nicaragua y Centroamérica, que habrían generado pérdidas económicas y problemas de reputación.

Integración entre herramientas y trazabilidad

El valor real llegó al **vincular resultados**: los pipelines CI (GitHub Actions) ejecutaban pruebas unitarias, de integración y UI; los resultados se sincronizaban con TestLink vía API, garantizando que **cada ejecución quedara asociada a su requisito** y a la versión del código que la generó. Esto permitió reproducciones fiables, auditoría y métricas longitudinales (defectos por módulo, tiempo medio de detección, cobertura). La guía práctica sobre TestLink facilitó la correcta instrumentación del flujo de pruebas.

Resultados cuantitativos y cualitativos

Después de 12 meses de adopción rigurosa:

- **Reducción del 70% en defectos críticos** reportados en producción.
- **Cobertura de pruebas:** >90% en componentes críticos (unitarias + integraciones).
- **Tiempo de validación** por release: de 10 días a menos de 24 horas (gracias a automatización y pipelines).
- **Menos reclamos operativos** en clientes de Nicaragua (reducción significativa de tickets relacionados a conciliaciones y pagos).
- **Auditoría y cumplimiento:** trazabilidad que soportó requerimientos contractuales y permitió pruebas de aceptación formales con clientes.

Lecciones prácticas y recomendaciones

1. **Diseñar el plan de pruebas como producto:** protocolos, tablas de decisión y suites en TestLink son artefactos del proyecto.
2. **Combinar técnicas:** usar caja negra para requisitos, caja blanca para lógica crítica (p. ej. finanzas), y tablas de decisión para reglas complejas.
3. **Automatizar donde importe:** flujos de negocio críticos deben tener pruebas UI automatizadas (Selenium) + pruebas de API integradas.
4. **No subestimar entornos regionales:** formatos de fecha, monedas y regulaciones locales exigen casos de prueba específicos.
5. **Instrumentar métricas y mantener evidencia:** TestLink ofrece la trazabilidad que hace auditables las decisiones de release.

Preguntas para Análisis y Discusión.

1. ¿Por qué es fundamental definir un plan de pruebas antes de automatizar?
2. ¿Qué diferencia existe entre las técnicas de caja negra y caja blanca, y cuándo aplicar cada una?
3. ¿Cómo contribuyen Selenium y TestLink al aseguramiento de la calidad en proyectos multilenguaje?
4. ¿Qué papel cumplen las métricas de cobertura y defectos en la mejora continua?
5. ¿Qué desafíos enfrentan los equipos distribuidos al aplicar pruebas de integración?
6. ¿Por qué las pruebas de regresión son críticas en entornos ágiles?
7. ¿Qué ventajas ofrece el software libre en comparación con herramientas comerciales de testing?
8. ¿Cómo puede la complejidad ciclomática ayudar a priorizar módulos para las pruebas?
9. ¿Qué estrategias podrían aplicarse para combinar pruebas manuales y automáticas eficazmente?

10. ¿Qué riesgos implica una baja trazabilidad entre requisitos, casos y resultados?